Yash Shah

Seattle, WA | yashsachinshah99@gmail.com | linkedin.com/in/yash2705 | github.com/yash2705

EDUCATION

North Carolina State University, Raleigh, NC — August 2024 – May 2026
M.S. Computer Science — GPA: 4.0/4.0
Coursework: Operating Systems, Software Security, DevOps, Generative AI, Software Architecture & Design

K J Somaiya College of Engineering, Mumbai, India — September 2020 - June 2024
B.S. Computer Engineering (Minor in Data Science) — GPA: 3.8/4.0
Coursework: Computer Networks, Database Management, Object Oriented Programming, Machine Learning

TECHNICAL SKILLS

- Programming Languages: Java, C/C++, JavaScript, TypeScript, Go, Python, HTML, CSS
- Frameworks: Spring Boot, React, Node.js, Angular, Express.js, React Native, Django, FastAPI, Flask
- Databases: DynamoDB, MySQL, PostgreSQL, MongoDB, Firebase, Redis
- Tools: Git, AWS, Azure, GCP, GitHub Actions, Docker, Kubernetes, Terraform, Ansible, Jenkins, Heroku

PROFESSIONAL EXPERIENCE

Software Development Engineer, Amazon Web Services, Seattle, WA — June 2026 – Present
- Designed and launched to production a high-throughput service in **Java** on **ECS** backed by **DynamoDB**, using autoscaling and read-path warming to sustain **2,500 TPS** at **50ms p99 latency**.
- Built an event-driven ingestion pipeline using **SNS** and **SQS** for near-real-time updates and an **EventBridge**-triggered **Lambda** reconciler, with idempotent DynamoDB writes eliminating data drift from duplicate events.

Software Developer, Khayrallah Center for Lebanese Diaspora Studies, Raleigh, NC — August 2025 – May 2026
- Architected a scalable search system using **FastAPI** and **Elasticsearch**, deployed on **EC2** to index and retrieve **450,000+** Arabic document pages, with a **React.js** frontend featuring real-time text highlighting.
- Integrated distributed OCR processing pipelines with **Celery** and **Redis** to extract metadata from Arabic documents and store results in **S3**, reducing data processing time by **60%**.

Software Development Engineer Intern, Amazon Web Services, Seattle, WA — May 2025 – August 2025
- Engineered a graph-based validation engine in **TypeScript** to detect schema inconsistencies during build-time, preventing **97%** of runtime errors and saving **6 hours** of manual debugging time each week.
- Root-caused and fixed **2** long-standing visual bugs on high-traffic pages serving **1M+** users, tracing them to misconfigured TOML files flagged by the validation engine I built.

Software Engineer Intern, nVent, Mumbai, IN — January 2024 – July 2024
- Redesigned a legacy Excel-based reporting system in **Angular** by implementing asynchronous batch processing of XLSX files using **RxJS**, eliminating timeout errors and achieving **100%** reliable processing.
- Achieved **92%** test coverage by implementing unit and functional tests using **Karma**, **Jasmine**, **Gherkin** and **Cucumber**, cutting regression defects by half and establishing a robust CI-ready testing framework.

Software Engineer Intern, Adigos Soft Technologies, Remote — April 2022 – September 2022
- Designed a low-latency engine in **Java** and **Spring Boot**, applying event-driven architecture with **Kafka** to guarantee ordered, durable bid processing and eliminate race conditions in concurrent auctions.
- Scaled the persistence layer using **PostgreSQL sharding** and write-ahead logging (WAL) optimizations, enabling fault-tolerant recovery and **sub-100ms** bid latency under concurrent load.

PROJECTS

Distributed Key-Value Store with Consistent Hashing
- Implemented a fault-tolerant distributed key-value store in **Java** using consistent hashing and **gRPC** for inter-node communication, enabling horizontal scalability across **Kubernetes** clusters.
- Maintained read and write availability through simulated node failures via replication, leader election, and automatic rebalancing, validated with stress tests handling 5,000 concurrent requests.

Distributed Transaction Manager
- Engineered a fault-tolerant transaction manager in **Java**, extending two-phase commit (2PC) with **ZooKeeper-based leader election and crash recovery**, guaranteeing atomic multi-node writes across **PostgreSQL** shards.
- Enhanced scalability by integrating a **Kafka-backed write-ahead log**, enabling replayable commits and exactly-once guarantees under coordinator or participant failures, sustaining **50k+ TPS** in stress tests.

POSIX-Compliant File System
- Developed a POSIX-compliant file system in **C++** using **FUSE**, implementing inodes, block allocation, and journaling to ensure crash recovery and data integrity under unexpected shutdowns.
- Optimized read/write performance with client-side caching, lazy writes, and multi-threaded I/O, achieving **3× higher throughput** compared to a baseline NFS-style prototype.